L11: Three Algorithms: Probability to Practice

C. V. Jawahar

IIIT Hyderabad

February 12, 2026

Recap:

- Problems of interest:
  - Learn a function $y = f(\mathbf{W}, \mathbf{x})$ from the data.
    - (a) Classification (b) Regression
  - Learn Feature Transformations $\mathbf{x}' = \mathbf{W}\mathbf{x}$ or $\mathbf{x}' = f(\mathbf{W}, \mathbf{x})$
    - (a) Feature Normalization (b) PCA/Dimensionality Reduction
- Algorithms/Approaches:
  - (i) Nearest Neighbour Algorithm (ii) Linear Classification: $sign(\mathbf{w}^T\mathbf{x})$ (iii) Decide as ω_1 if $P(\omega_1|\mathbf{x}) \geq P(\omega_2|\mathbf{x})$ else ω_2.
  - Linear Regression: (a) closed form and (b) GD; Ridge and Lasso
  - Bayesian View; Normal Distribution; MLE and GMM
  - K Means; Naive Bayes and Logistic Regression
- Supervised Learning:
  - Notion of Training, Validation and Testing, Performance Metrics
  - Low rank data matrix and SVD; PCA and Eigen Faces
  - Notion of Loss Function, (eg. MSE), Regularization.
  - Role of Optimization, Convex and non-Convex optimization
  - Closed form solution, Gradient Descent, Eigen vector solns.

Recap of Probabilistic Perspective of ML

- Bayes Theorem
- Normal Distribution (Univariate and Multivariate)
- Decision Boundary and Discriminant Functions (shape, analytical form)
- Optimality of Bayesian Classifier
- MLE and Log Likelihood
- MLE for parameter Estimation
- MLE based interpretation of MSE (Linear regression)
- GMM for approximating complex multimodal distributions

Recap of GMM

1. Initialize the means μ_j, covariances Σ_j, and the mixing coefficients π_j, and evaluate the initial value of log likelihood.
2. E-Step: Evaluate the responsibilities

$$\gamma_k(x) = \frac{\pi_k \mathcal{N}(x|\mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x|\mu_j, \Sigma_j)}$$

3. M-Step: Re-estimate the parameters using the current responsibilities

$$\pi_j = \frac{1}{N} \sum_{i=1}^{N} \gamma_j(x_i); \;\; \mu_j = \frac{\sum_{i=1}^{N} \gamma_j(x_i) x_i}{\sum_{i=1}^{N} \gamma_j(x_i)}$$

$$\Sigma_j = \frac{\sum_{i=1}^{N} \gamma_j(x_i)[x_i - \mu_j][x_i - \mu_j]^T}{\sum_{i=1}^{N} \gamma_j(x_i)}$$

4. Evaluate log likelihood. If there is no convergence, return to step 2.

Problem of Clustering

Problem of Clustering: Partition a set into mutually exclusively and collectively exhaustive sub-sets.

$$A = C_1 \cup C_2 \cup \ldots \cup C_k \text{ such that } C_i \cap C_j = \phi$$

- Agglomerative and Divisive Clustering
  - **Agglomerative** Start with many small clusters and merge
  - **Divisive** Start with one big cluster and split
  - Eg. Single linkage algorithms using MST/Graph theory
- Role of Hashing
  - Find right hash functions (know about LSH)
  - Efficient O(N)?

K Means

Problem:

$$Minimize \sum_i \sum_k ||x_i - \mu_k||_2^2 \text{ where } x_i \in C_k$$

and μ_k is the mean of cluster C_k

Procedure:

1. Randomly Partition the set into K Subsets; and
2. Compute means of these subsets as $\mu_1 \ldots \mu_k$
3. Reassign every sample to the set where mean is the closest.
4. Repeat steps 2-4 until convergence.

Appreciate how each step (i.e., Steps 2 and 3) minimizes the objective.

K Means: Numerical Example

Problem 1: 10 Points in 1 D and K = 2

$$\{2, 15, 3, 16, 5, 1, 4, 18, 22, 20\}$$

Problem 2: 6 Points in 2 D and K = 2

$$\{A(1,1), B(1.5,2), C(3,4), D(5,7), E(3.5,5), F(4.5,5)\}$$

K Means: Discussions

- K Means and similarity with GMM
- K Means as EM
- Does the final answer depend on the initialization?
- What happens to the objective when you increase K?
- How do we find K?

Naive Bayes Classifier

Naive Bayes is popular for categorical or discrete data, though it can also be used for continuous features.

Bayes theorem

$$P(Y|X) = \frac{P(X|Y) \cdot P(Y)}{P(X)}$$

In a classification problem, X may have many attributes, (say d attributes) and Y may be integer, (say the number of classes)

We will not have even one occurrence of all possible combinations for X (i.e., if attributes are binary, our data set could be much smaller than 2^d)

We can not reliably compute $P(X|Y)$ from small amount of data we have.

Conditional Independence

Features $x_1, x_2, \ldots, x_d$ are conditionally independent given class y if

$$P(x_1, x_2, \ldots, x_d | y) = \Pi_{j=1}^{d} P(x_j | y)$$

Note: They are not assumed independent overall, they are independent after we know the class.

Assume conditional independence

$$P(X^1 X^2 \ldots X^d | Y) = \Pi_{i=1}^{d} P(X^i | Y)$$

This reduces to sample requirements and make Bayesian classifier practical (though naive!!)

Logistic Regression

Background: Logistic Function or Sigmoid

$$g(x) = \frac{1}{1 + e^{-x}}$$

Q: Plot this function

Problem: Our problem is to design a binary classifier (i.e., y = 0 or 1) that also give probability/confidence in each class.

$$p(y = 1|\mathbf{x}) = g(\mathbf{w}^\mathbf{T}\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\mathbf{T}\mathbf{x}}}$$

$$p(y = 0|\mathbf{x}) = g(-\mathbf{w}^\mathbf{T}\mathbf{x}) = \frac{1}{1 + e^{\mathbf{w}^\mathbf{T}\mathbf{x}}}$$

Q: Do they sum up to one?

Q: What is the binary classification rule?

Logistic Regression: Objective

Likelihood of the data is given by:

$$l(\mathbf{w}) = \prod_{i=1}^{N} p(y_i|\mathbf{x_i}; \mathbf{w})$$

Looking for $\mathbf{w}$ that maximizes the likelihood

$$\mathbf{w}_{MLE} = \arg\max_{\mathbf{w}} \; l(\mathbf{w}) = \arg\max_{\mathbf{w}} \; \prod_{i=1}^{N} p(y_i|\mathbf{x_i}; \mathbf{w})$$

Logistic Regression

Let us look at the loss function with $y_i \in \{0, 1\}$.

$$p(y = 1|\mathbf{x}) = g(\mathbf{w}^T\mathbf{x}) \text{ and } p(y = 0|\mathbf{x}) = 1 - g(\mathbf{w}^T\mathbf{x}) \text{ or compactly}$$

$$p(y|\mathbf{x}) = g(\mathbf{w}^T\mathbf{x})^y(1 - g(\mathbf{w}^T\mathbf{x}))^{1-y} \tag{1}$$

Assuming independence in the data/samples, likelihood is

$$l(\mathbf{w}) = \prod_{i=1}^{N} g(\mathbf{w}^T\mathbf{x_i})^{y_i}(1 - g(\mathbf{w}^T\mathbf{x_i}))^{1-y_i}$$

$$\log l(\mathbf{w}) = \sum_{i=1}^{N} \Big[y_i \log(g(\mathbf{w}^T\mathbf{x_i})) + (1 - y_i)\log(1 - g(\mathbf{w}^T\mathbf{x_i}))\Big]$$

Take negative, we get the "binary cross entropy loss" (where $z_i = w^T x_i$)[1]

$$\mathcal{L}(\mathbf{w}) = -\sum_{i=1}^{N}[y_i \log g(z_i) + (1 - y_i)\log(1 - g(z_i))]$$

[1] Binary Cross Entropy $L = -[y \log p + (1 - y)\log(1 - p)]$

LR with $y_i \in \{\pm 1\}$

We can also rewrite the objective with the $y \in \{-1, +1\}$ convention.

$$p(y = +1|\mathbf{x}) = g(\mathbf{w}^T\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T\mathbf{x}}}$$

$$p(y = -1|\mathbf{x}) = 1 - g(\mathbf{w}^T\mathbf{x}) = \frac{1}{1 + e^{\mathbf{w}^T\mathbf{x}}}$$

We can combine both into single expression as

$$p(y_i|\mathbf{x_i}) = \frac{1}{1 + e^{-y_i\mathbf{w}^T\mathbf{x}_i}}$$

Assuming independence, the likelihood is

$$\prod_{i=1}^{N} \frac{1}{1 + e^{-y_i\mathbf{w}^T\mathbf{x_i}}}$$

Then the negative log likelihood is

$$\sum_{i=1}^{N} \log(1 + e^{-y_i\mathbf{w}^T\mathbf{x_i}})$$

Multiclass and SoftMax

We know that: $p(y=1|\mathbf{x};\mathbf{w}) = \frac{e^{\mathbf{w}^T\mathbf{x}}}{1+e^{\mathbf{w}^T\mathbf{x}}}$ and

$$p(y=0|\mathbf{x};\mathbf{w}) = 1 - p(y=1|\mathbf{x};\mathbf{w}) = \frac{1}{1+e^{\mathbf{w}^T\mathbf{x}}} = \frac{e^{-\mathbf{w}^T\mathbf{x}}}{1+e^{-\mathbf{w}^T\mathbf{x}}}$$

Numerator is the weight/confidence of the sample being that class and denominator is the sum of weights for all classes. This allows us to extend to the multiclass setting as

$$p(y=c|\mathbf{x};\mathbf{w_1},\mathbf{w_2},\mathbf{w_K}) = \frac{e^{\mathbf{w_c}^T\mathbf{x}}}{\sum_{i=1}^{K} e^{\mathbf{w_i}^T\mathbf{x}}}$$

Note that the sum of probabilities across all classes sum upto one. Finally a sample is classified into

$$\arg\max_i p(y=i|\mathbf{x})$$

This is also called popularly as **softmax**. Very popular in the modern deep learning architectures.

Problem

Why does A Simple Loss Function not used?
One can get tempted to use the same loss function here as:

$$J(\mathbf{w}) = \frac{1}{2N} \sum_{i=1}^{N} (g(\mathbf{w}^T \mathbf{x}_i) - y_i)^2$$

Unfortunately, this is a poor choice of loss function. This is non-convex.

- Question: Prove or Verify that.

Note: we assumed here $y_i \in \{0, 1\}$

Problem

We know that:

$$g(z) = \frac{1}{1 + e^{-z}}$$

$$tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Plot these:

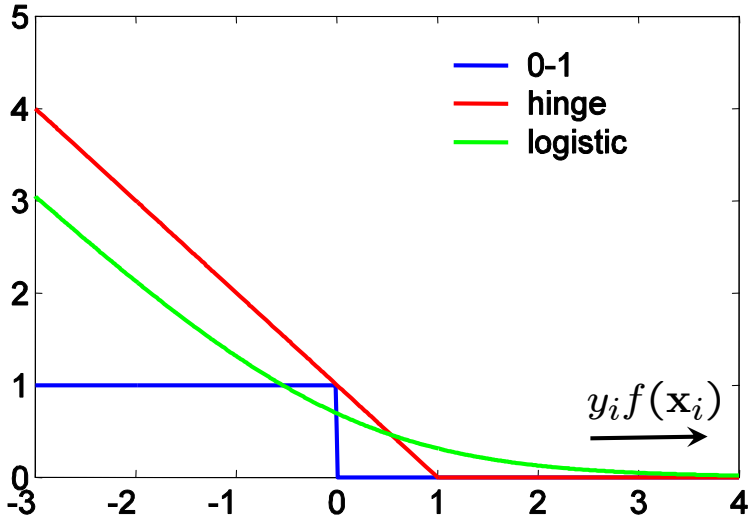
1. Prove that
$$g'(z) = g(z)(1 - g(z))$$
2. Verify $tanh(z) = 2 \cdot g(2z) - 1$
3. Express derivative of tanh in terms of tanh

Discussion Point

Note: Hinge loss $L = \max(0, 1 - yf(x))$

SVM:

$$\min_{\mathbf{w}} C \sum_{i=1}^{N} \max(0, 1 - y_i f(\mathbf{x_i})) + ||\mathbf{w}||^2$$

Logistic Regression

$$\min_{\mathbf{w}} \sum_{i=1}^{N} \log(1 + e^{-y_i f(\mathbf{x_i})}) + \lambda ||\mathbf{w}||^2$$